[Document title]

[DOCUMENT SUBTITLE]

Authors
DESALES UNIVERSITY | [COMPANY ADDRESS]

Software Design Document Specification Template

The Software Design Specification (SDS) sections provide you with guidelines related to the structure and the contents of SDS document. The Software Design Specification document includes at least these sections.

For the project, your team may have good reasons for wanting to deviate from this proposed outline. If a section is not applicable in your case, do not delete it; instead, give the topic heading and write "Not applicable".

You will note that there is some overlap in the content between different documents (i.e. the User Requirements Specification Document and the Software Design Specification Document.) This redundancy allows each document to stand on its own.

ONLY THE SECTION TITLES COLORED IN ORANGE ARE REQUIRED TO BE COMPLETED.

DO NOT DELETE THE SECTIONS YOU ARE NOT COMPLETING AS THEY ARE A PART OF THE DOCUMENT

Contents

1 Introduction 3
1.1 Purpose of this document 3
1.2 Scope of the development project 3
1.3 Definitions, acronyms, and abbreviations 3
1.4 References 3
1.5 Overview of document 3
2 System architecture description 3
2.1 Overview of modules / components 3
2.2 Structure and relationships 3
2.3 User interface 3
2.4 User interface issues 3
3 Detailed description of components (ONLY 2 ARE REQUIRED) 3
3.1 X Component (or Class or Function ...) 3
3.2 Y Component (or Class or Function ...) 4
4 Reuse and relationships to other products 4
5 Design decisions and tradeoffs 4
6 Pseudocode for components 4
7 Appendices (if any) 4
Software component template for section 3 4

1 Introduction

1.1 Purpose of this document

Full description of the main objectives of the SDS document.

1.2 Scope of the development project

This will be similar to what was written in the SRS.

1.3 Definitions, acronyms, and abbreviations

Be sure to alphabetize!

1.4 References

This section will include technical books and documents related to design issues. Be certain that the references you give are complete and in the appropriate format.

1.5 Overview of document

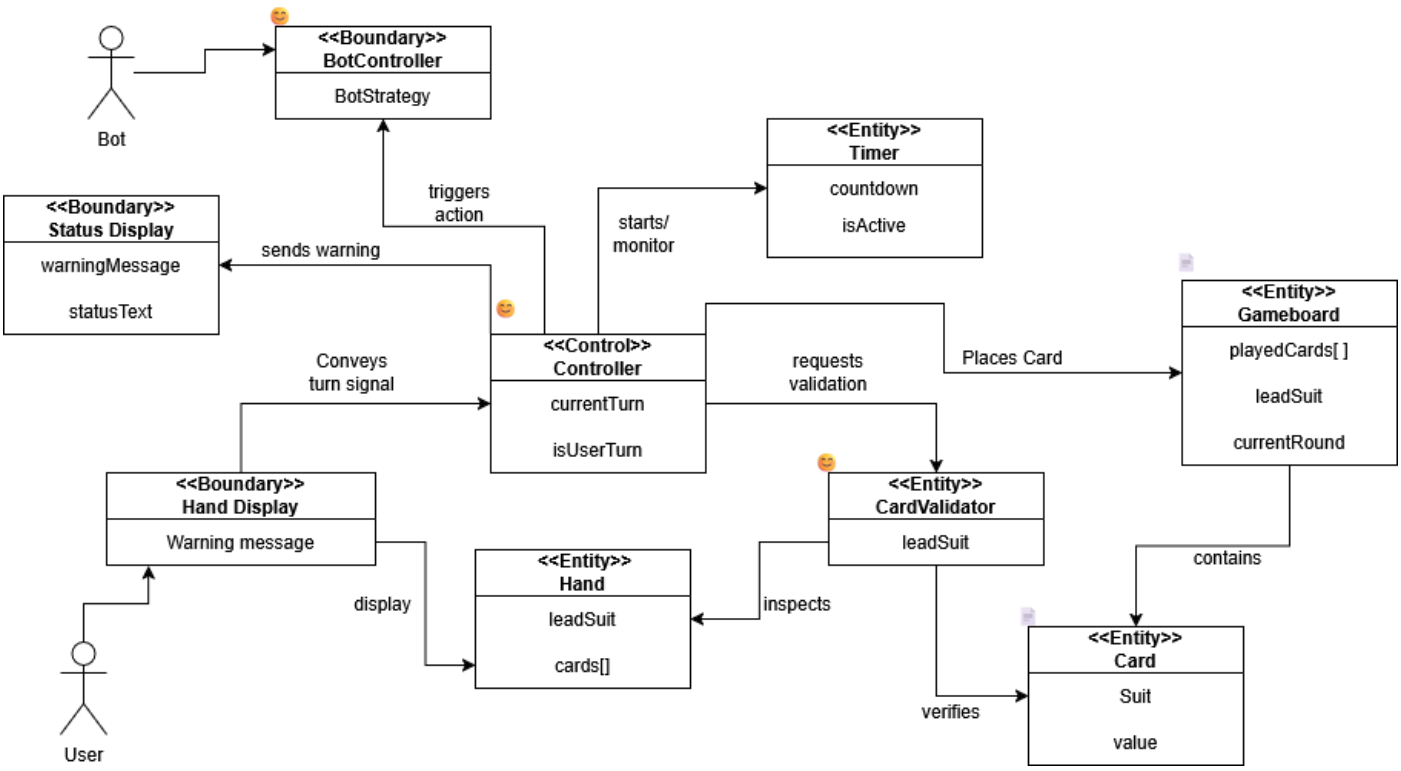
A short description of how the rest of the SDS is organized and what can be found in the rest of the document. This is not simply a table of contents. Motivate and briefly describe the various parts!

2 System architecture description

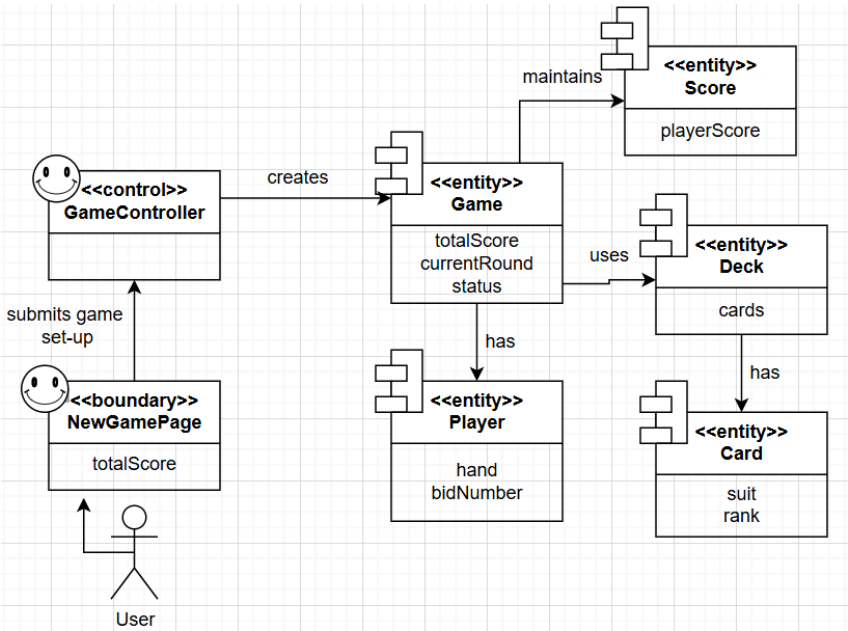
2.1 Overview of modules / components

This subsection will introduce the various components and subsystems.

Domain model for: UC-1 **Card Selection**



Domain model for: UC-4 **NewGame**



2.2 Structure and Relationships

Make clear the interrelationships and dependencies among the various components. A simple finite state machine can be useful in demonstrating the operation of the product. Include explanatory text to help the reader understand any charts.

UC-1 **Card Selection**

Responsibility Description	Type	Concept Name
Ensures that the card chosen by user follows the rules	D	CardValidator
Displays the score, players hand, and cards at play	D	GameBoard
Monitors overall game activity. This includes warning signals to player, who's turn it is (bot or user), and calculating the score	D	Controller
Interface that appears between rounds, updating the user on how they did during the previous round	D	Status Display
This is a game that cannot be played as a single person's game. The bot controller will take the place of the other players who are needed to play the game.	D	BotController

UC-4 **New Game**

Responsibility Description	Type	Concept Name
Accepts the user's game setup input and sends it to the system to begin a new game	D	NewGamePage
Manages the overall flow of the game, receives setup information, and creates a new game session	D	GameController
Stores and manages the current game state, including the total score, current round, and game status	K	Game
Stores player specific values such as hand and bid number	K	Player
Scores and maintains each player's score throughout the game	K	Score

Represents the collection of cards used during the game	K	Deck
Represents an individual playing card with attributes such as suit and rank	K	Card

2.3 User Interface

Complete design with user interactive windows, error messages, etc. Include all windows and messages that will be viewed by the user and also the ones that the user will be interacting with.

Our application will consist of 5 different user interactive HTML screens and 4 different user interactive pop-ups. The initial HTML screen the user shall interact with will be the *Home* screen which will welcome the user to our application Pitch Deck. This screen will contain the "New Game" button, allowing the user to be redirected to the *Maximum Points* screen when clicked.

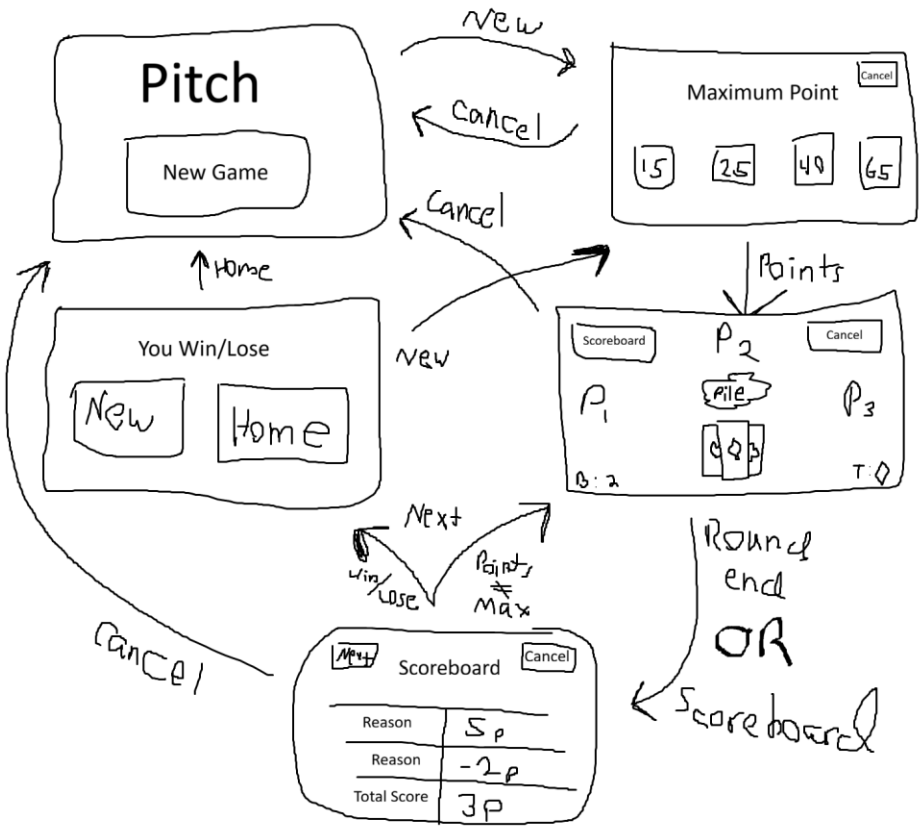
The *Maximum Points* screen will ask the user to choose the winning point value for their new game. This screen will contain 4 buttons, each representing a different maximum point value that either the player or one of the computer-generated bots will have to reach before they win the game with the points being 50, 75, 100, or 150. If the user clicks one of the 4 number buttons, that value will become the maximum point value for that game of Pitch Deck and the user will be redirected to the *Game* screen. The user can also click the "Cancel" button where an interactive pop-up will appear, asking the user if they want to proceed and warning them that all game progress will be lost. If the user clicks the "Yes" button, they will be redirected back to the *Home* screen. If the user clicks "No", the pop-up will close.

The *Game* screen will contain a card deck (face-down), a card pile (face-up), 4 card hands (3 face-down, 1 face-up), three computer-generated bots to act as other players, the user's bid, and that round's trump suit. First, the system will shuffle the cards before dealing them to all 4 players, 7 per player. After the user has 7 cards, an interactive pop-up will appear asking the user to choose their bid for that round where the user will have an input box to choose a bid of 0 to 18. Once the user selects their bid, one of two events will occur. If the user has not chosen the highest bid or is not the dealer if all players have chosen a bid of 0, that round's trump suit will be randomly chosen. If the player has either the highest bid or is the dealer with all players choosing a bid of 0, an interactive pop-up will appear asking the user to choose the trump suit for that round where the user will have 4 buttons to choose containing the suits diamond, club, heart, or spade. Once the trump suit is chosen, an interactive pop-up will appear asking the user if they would like to discard any non-trump suit cards in their hands. The user can then interactive <u>only</u> with the non-trump suit cards and by double-clicking them, they will be discarded and replaced with shuffled new cards. All trump suited cards will not be interactable during this phase. When the user is either finished with or wish to skip the discard phase, the pop-up will contain a "Continue" button that, when clicked, will continue with the game Pitch Deck. When the round starts, the user will have three interactable features. The first feature is the "Scoreboard" button which will direct the user to the *Scoreboard* screen which will display the current scoreboard for that round. The user can click the "Next" button to be redirected back to the Game screen to continue the round, or the user can click the "Cancel" button where the user will be redirected to the *Home* screen after its associated pop-up. The second feature is the "Cancel" button where the user will be redirected to the *Home* screen after its associated pop-up. The third feature is the user's cards which will <u>only</u> be interactable during the user's turn, and <u>only</u> the viable cards will be interactable. The cards that can't be played during the user's turn and all the user's cards not during the user's turn will not be interactable.

The *Scoreboard* screen will display the scoreboard for the user's current game of Pitch Deck. The user will automatically be directed to the *Scoreboard* screen after each round is over where the scoreboard will display the points the user has gained/lost during that round, the reasoning as for why, the other players' scores for that round, and all players' overall scores for that game. This screen will contain two buttons: the "Cancel" button where the user will be redirected to the *Home* screen after it's associated pop-up and the "Next" button which has alternate functions. If none of the 4 players have a total score that is equal to or greater than the maximum score value, the "Next" button will redirect the user to the *Game* screen where a new round of Pitch Deck will begin. If one of the players has a total score that is equal to or greater than the maximum score value, the user will be directed to the

Win/Lose screen. If two or more players have the same score that is equal to or greater than the maximum score value, the user will still be directed to the *Win/Lose* screen.

The *Win/Lose* screen will display one of two messages for the user. If the user had a total score that was equal to or greater than the maximum score value, the message “You Win” will be displayed. If the user didn’t have a total score that was equal to or greater than the maximum score value, the message “You Lose” will be displayed. If two or more players had the same score that is equal to or greater than the maximum score value, if the user was among them, the message “You Win” will be displayed, otherwise the message “You Lose” will be displayed. This screen will contain two buttons. The first button is the “New Game” button which will direct the user to the Maximum Points screen. The second button is the “Cancel” button which will redirect the user to the *Home* screen after its associated pop-up.



2.4 User Interface Issues

The goal of our team’s user interface is to make it user friendly as possible. When logging onto the site, you will be greeted with a home page, where the user will have the option to start a game or look at a rule book to better understand the game. A first-time user will find this feature helpful, before diving into the game. Once the user decides they are ready to start a game, they can exit the rule book and click the button “start game.” Once they have done this, they will be prompted to enter what they would like to play their game to: 50, 75, 100, or 150. The user will then be brought to the dealing screen where the game will be played.

3 Detailed Description of Components (At Least 8)

3.1 Game Storage

Identification	The *Game Storage* will be located in the database unit of the system.
Type	A database

Purpose	The purpose of the game storage component is to provide persistent data storage to all necessary information pertaining to the gameplay of Pitch Deck including player information, the game state, each round's inputs, and the scores as well as the related scoring rules. Without all this information being within a persistent storage unit that is readily accessible to the client application, the functions related to the gameplay won't work and users won't be able to use this application to its fullest extent. The derived requirements for this storage unit is that data integrity remains secure, that all information storages is easily retriable by the game's system, and that the stored data remains synronized across all rounds within a single instance of a game.
Function	The functionality of the game storage component is to store and manage the persistent data storage of all the information pertaining to both the gameplay itself and the individual rounds that make up each game itself. The game storage component needs to store all information related to the 4 players involved in each game, the state of each game, the information that makes up each round of the game, and the scores that the players earned at the end of each round across the entire instance of a game. While this component doesn't produce any outputs itself, the information it stores is needed for the execution of alternate functions like the calculation of the score and the determination of the winner. The game storage component will also have the ability to update any existing information it carries whenever necessary and retain that overridden information in situations where: - A new instance of the game is created. - A new instance of a round within a game is created. - The scores for each player after each round is updated.
Subordinates	The internal structure of the game storage component will be constructed and deployed as followed for the following reasons stated: - A managar will need to be developed to store and retrieve objects that pertain the the game instances themselves so the gameplay can run as intended (GameDataManager). - A managar will need to be developed to store and manage player information in the system for each instance of the game (PlayerDataManager). - A managar will need to be developed to store and retrieve any information retaining to the scores of the players each round throughout the game (ScoreDataManager). - A Data Access Layer (DAL) will need to be developed to act as an intermediary between the client application and the database to provide communication.
Dependencies	The game storage component will be used by will several of the system's other components. All interactions between this storage and other components will occur whenever game data must be saved or retrieved: - The *Database system* will be depended on by the game storage component in order to use the persistent game data. The storage will interact with this component throughout the running of this application. - The *GameController* will store and retrieve all game information whenever necessary. The storage will interact with this component throughout the running of this application. - The *Score Calculation Module* will update the stored scores for all players at the end of each round throughout each game with their new values. The storage will interact with this component at the each of each round.
Interfaces	Because the game storage component is not a component not meant for user interaction, only the recipient of user input; most of the interfaces shall be internal, with the only external interface being for the database. The interfaces necessary for this module are as follows: - **saveGame()**: This interface will be used by the *database system* and *GameController* and other components to store either a new game instance or updated game state in

	persistent storage. In order to store new game states, this interface will need the game's state, the player's data, data on the scores, and the round's data. Possible errors would be if any of the needed information is absent or if this interface can't make a connection with the database. - **updateScoreRecord()**: This interface will be used by the *database system* and *GameController* and other components to update the score data stored in the database. In order to update the players' scores, this interface will need the players' ID and the new value it's replacing the old score with. A possible error would be if any of the needed information is absent. - **Database Interface**: This interface will be used so the game storage component can make a connection with the *database system* by using various communication methods such as SQL queries and database drivers or ORM calls.
<u>Resources</u>	Because the Pitch Deck application runs is a single-player, server-less game that runs locally, the game storage component relies on software resources in the client application: - Local Storage: The local storage provides persistent storage for game data such as player information, player scores, and information about each round which will be stored in serialized objects. - RAM: The RAM stores all active game state information during gameplay, including the *Game* object, *Player* objects, deck information, score data, and round status. This storage allows for access and updating of the game state during the game. - Input/Output Services: Various I/O services are used to read from and write to the storage when saving or retrieving game state. - Application Data Structures: Internal structures such as lists, arrays, or dictionaries are used to manage the game state such as the player information, player scores, and information about each round.
<u>Processing</u>	The game storage component is responsible for managing the storage of the application and retrieval of the game state. When a new game is created, the *GameController* generates a *Game* object that contains the initial configuration of the game, including the user selected maximum winning value, player objects, score record with a default value of 0 for all players, and a shuffled deck of cards. This information is stored in the application's internal data structures and becomes an active game state. Throughout the gameplay, the game storage component maintains the current state of the game by keeping track of all relevant game data objects. These include the current round number, player hands, cards remaining in the deck, and the scores for each player. When the game state changes whenever a round is completed or the scores updated, the correlated data structures are updated in response. The game storage component also provides controlled access to game data for other parts of the system. When another component, such as the *GameController*, requires information about the game state, the game storage component retrieves the requested data from the internal structures and provides it to the requesting component. This ensures that all components operate on the updated game state.
<u>Data</u>	For the game storage component, all data is stored in a persistent database and accessed through communication with the DAL. The internal data structures for the storage will include: - The information across the entire game (Game Record). This data structure will need to know each player's total score (playerScore) so the system will know the players' starting each round and when to declare a winner, the current round (currentRound) so the system will know the information perdinate to the current round, and the game state (status) so the system will know if a new round or new game instance is to be configured. - The information across the entire game (Player Record). This data structure will need to know each player's ID number (playerID) so the system will know which player they

	are working with and each player's card hand (hand) so the system will know the cards still left in the deck ready to be distributed. - The scoring result for each round played by the user (Score Record). This data structure will need to know each player's ID number (playerID) so the system will know which player they are working with and the amount of points each player gained during that round (playerScore) so the system will know the players' starting each round and when to declare a winner.

3.2 Score Calculation Module

Identification	The *Score Calculation Module* will be located in the data processing unit of the system.
Type	A module
Purpose	The purpose of the score calculation module is to perform the calculation of the score for the user as well as the 3 computer-generated bots acting as other players during each individual round, when each round has concluded, and when the game has finished. This is necessary for both the system and the user can keep track of the scoring through each round and the game in total so the system will be able to conclude when the game must end and if to declare the user as the winner or loser and so the user can see their score throughout the game and to know whenever they have won or lost. The derived requirements for this module are the automatic calculation of the scoring through the game, the prevention of any errors in said calculations, and for the scoring to remain consistent throughout each round and each new game initialized.
Function	The functionality of the score calculation module is to calculate the score of the user as well as the 3 bots during each round, at the end of each round, and at the end of the game. In order to work properly, this module will need to know the maximum winning value the game is playing until so it can determine if the user has won the game or not and the bidding value for each player so it can calculate the score each player obtains per round. The calculations themselves will be stored in the backend, but the outputs of said calculations (including each point gained/lost, the reasoning why, and the final score) will be visually displayed in the Scoreboard HTML page. The calculations for the points gained are as follows: - If the player wins number cards from 2-9, they gain 0 points. - If the player wins a jack card, they gain 1 point. - If the player wins a queen card, they gain 2 points. - If the player wins a king card, they gain 3 points. - If the player wins an ace card, they gain 4 points. - If the player wins a number card of 10, they gain 10 points. - If the player doesn't gain the amount of points they bid, they lose the amount of points that they bid.
Subordinates	The internal structure of the score calculation module will be constructed and deployed as followed for the following reasons stated: - An evaluator will need to be developed for the scoring of each round to determine the amount of points earned from each completed round (RoundScoreEvaluator). - An updater will need to be developed to apply the calculated points gained each round to the Score entity (RoundScoreUpdater). - A checker will need to be developed to determine if a bidding player, both the user and the computer-generated bots, have met their bid in order to determine if they win or lose points (BidChecker).

	- A checker will need to be developed to determine whether a player's score is equal to or exceeding the maximum winning value to end the game and determine the wining player (GameWinnerChecker).
<u>Dependencies</u>	The score calculation module will be used by and interact will several of the system's components. All interactions between this module and the other components will occur after a round is completed: - The *GameController* will call the score calculation module after each round to process the scoring for each of the players, both the user and computer-generated bots. The module will interact with this component first to calculate the scoring for that round. - The *Player* will provide the bid number that is required for the scoring to determine whenever or not the score for a player gained that round will be won or the player will lose the points they bid. The module will interact with this component second to gain the stored information necessary for determining the scores for each player. - The *Game* will provide the maximum winning value that a player's score must meet in order for the game to end as well as any alterative information about the game. The module will interact with this component third to gain the stored information necessary to see if the game is to end. - The *Score* will store all the updated score values which will be displayed on the Scoreboard HTML page. The module will interact with this component fourth to store the calculated scoring information (if the game is still ongoing).
<u>Interfaces</u>	Because the score calculation module is not a component meant for user interaction, only the recipient of user input; all the interfaces shall be internal, some of which do display external outputs. The interfaces necessary for this module are as follows: - **calculateRoundScores()**: This interface will be used by the *GameController* to call on the score calculation module to calculate the scoring for all the players, both the user and the computer-generated bots, for each individual round. In order to calculate the scores per round, this interface will need each player's ID, each player's bid that round, and the points each player gained that round. Possible errors would be if any of the needed information is absent or if the calculation failed. - **updatePlayerScores()**: This interface will be used by the *GameController* to apply the score calculated that round for each player to the *Score* entity, adding it to the player's previous score to create a new updated score. In order to update the scores per round, this interface will need each player's ID, each player's score that round, and each player's score that game. Possible errors would be if any of the needed information is absent or if any of the players can't be found. - **checkGameWinner()**: This interface will be used by the GameController to determine if the player's, either the user or the computer-generated bots, score is equal to or exceeding the maximum winning value for that game. In order to determine the winner, this interface will need each player's ID, each player's score that game, and that game's maximum winning value. Possible errors would be if any of the needed information is absent.
<u>Resources</u>	Because the Pitch Deck application runs is a single-player, server-less game that runs locally, the score calculation module relies on software resources in the client application: - CPU execution time: The CPU performs the calculations required to evaluate the round results and player scoring. Whenever a round ends, the processor calculates and updates the scores for each player. - RAM: The RAM stores temporary data structures used during the score evaluation process including round results, player bids, and calculations before the final scores are sent to the *Score* entity.

	- Game State Objects: The module requires access to game objects such as Game, Player, and Score to provide the data needed to perform the score calculations and store the updated results. - Math Libraries: Math libraries are necessary to perform the basic arithmetic and logical operations that the score calculation module requires to evaluate the score results for the players. - Local Storage: Score information is stored and updated within the application's local storage which maintains the score during gameplay and allow scores to be referenced by other components like the *GameController* or Scoreboard.
Processing	The score calculation module executes whenever a round of the game is completed. The *GameController* calls on the process by providing the results of the round, including each player's bid and the points each player gained that round. The module first retrieves the bid information of how much each player bids at the beginning of the round. Then, it evaluates the cards won during the round to determine how many points were gained by each player. The scoring rules of the game are applied to determine the total value of the won cards to find the points each player gained that round. After the gained points are determined, the module compares the number of points gained to their bid amount. If the player meets or exceeds their bid, the earned points are added to that player's total score. If the player hasn't met their bid, the bid value is subtracted from the player's score. Once the score is calculated, the module updates the *Score* entity with each player's new overall score where the score values are stored. After updating the score, the module checks whether any player has reached or exceeded the maximum winning value specified when the game was created. If the winning value has been reached, the *Game* status is updated to show the game has been completed, and a winner determined. If the winning value has not yet been reached, the *GameController* initializes the next round of the game.
Data	For the score calculation module, all data involved in the scoring of the players' performances are stored in the *Score* entity and consistently maintained across all the game's rounds. The internal data structures for the scoring will include: - The score for each player across the entire game (Score). This data structure will need to know each player's ID number (playerID) so the system will know which player they are working with and each player's score (playerScore) so the system will know if a player wins the game. - The scoring result for each round played by the user (Round Result). This data structure will need to know the bid amount that each player indicated at the beginning of that round (bidAmount), and the amount of points each player gained during that round (pointsGained).

3.3 CardValidator

Description	Ensure that the card selected by the player follows game rules.
Responsibilities	• Check if the card is valid for the current turn • Enforce suit-following rules • Prevent illegal plays
Inputs	• Selected card • Current game state • Player hand
Outputs	• Boolean (valid/invalid) • Error message if invalid

Interfaces	• validateCard(card, gameState)
Dependencies	• GameController • Player

3.4 DeckManager

Description	Responsible for managing the deck of cards, including shuffling, dealing, and tracking remaining cards.
Responsibilities	• Create and initialize deck • Shuffle cards • Deal cards to players • Track used/remaining cards
Inputs	• Game start signal • Number of players
Outputs	• Distributed hands • Remaining deck state
Interfaces	• shuffleDeck() • dealCards(numPlayers)
Dependencies	• None (self-contained logic component)

3.5 Bid Manager

Description	The Bid Manger ensures the player is making a bid within the 1 – 4 range. It will also check the users' input to see if that bid has already been made. This is important because no two users can make the same bid.
Responsibilities	- Prompt user to make bid when it is their turn - Prompt user to choose bid: 1, 2, 3, 4, Pass - Ensure player to left of dealer starts bid - If no players make a bid dealer must take a bid of 2
Inputs	- The user has started round, and needs to make bid
Outputs	- The winner of bid will decide trump suit - The winner is also at risk of losing the amount they bid if they do not earn the same or more bid points
Interfaces	- bidManager() - makeBid() - noBid()
Dependencies	- Users

3.6 Dealer Manger

Description	The Dealer Manger will ensure that every round a new dealer is chosen. The first round the dealer will be chosen at random. Then, in the following rounds, the dealer will move the person to the left.
Responsibilities	- First round, choose a random player to be dealer - Every round the dealer's responsibility will be moved to the left.
Inputs	- New game/round has started
Outputs	- Dealer is chosen (important for distribution of cards, bidding, etc.)
Interfaces	- newDealer() - leftDealer()
Dependencies	- None (self-contained logic component)

3.7 Player Manger

Description	The Player Manger will ensure that the same players are present for each game. It will also be responsible for ensuring that the players stay in the same order/seating position for every round.
Responsibilities	- Ensure that the seating arrangements of players stay the same - Ensure the same players are present for the entirety of the game
Inputs	- New game started
Outputs	- Game consists of same players while its played - Players are on the same seating arrangements while the game is played
Interfaces	- playerManager() - seatArrangement() - samePlayers()
Dependencies	- User trying to join in the middle of a game

3.8 Card Mangager

Description	Card Manager will ensure that players cannot throw cards out of turn. This is crucial to the game rules, because the first card of every sub-round matter is important.
Responsibilities	- The player the won the bid, or pervious sub round will throw the first card - Players turn go a clockwise rotation. So, the person to the left of the winner of the bid, or the pervious sub round will go next. - Players will not be able to throw a card until it is their turn.
Inputs	- Game started - Bid was made
Outputs	- Ensure game rules are followed/players go when it is their turn
Interfaces	- cardManager()
Dependencies	- None (self-contained logic component)

3.5 Z Component (or Class or Function ...)

Examples of a component are:

1) Database
2) Server
3) Client application
4) Search Application
5) Etc.

4 Reuse and relationships to other products

For teams doing enhancement work, reuse is an important issue. Most enhancement work should focus on extending, rather than replacing, the design and product development from earlier semesters. For teams doing new development, reuse can also be an important strategy. In some cases, there is freeware that could be incorporated. In other cases, there are existing modules or classes that could be adapted. Another possibility is the use of special tools that produce open source results and thus permissible under the terms of this course.

This section should include the following subsections as appropriate:

- How reuse is playing a role in your product design
- How reuse is playing a role in your product implementation (and the motivation for changes)
- If you are not reusing material that is available, then give motivation for why it is being thrown out.

5 Design decisions and tradeoffs

5.1 Bot-Based Multiplayer Architecture

Pitch Deck is a card game that requires four players. Because the application targets a single human user, the team needed to decide how to populate the remaining three seats.

Decision adopted:

Three computer-controlled bots (managed by BotController) replace the absent human players. The BotController boundary concept is triggered by the Controller once the user's turn ends, keeping turn flow consistent regardless of whether a seat is human or automated.

Alternatives considered:

- Online multiplayer with real human players.
- A local hot-seat mode allowing multiple humans to share one device.

Rationale for rejection:

- Online multiplayer would require a backend server, user authentication, session management, and network synchronisation — scope well beyond the project timeline.
- Hot-seat mode degrades the single-player experience and complicates the UI flow significantly.

Tradeoff accepted:

Bot strategy is currently simplified (deterministic lowest-valid-card logic). This means experienced players may find the game too easy, but it keeps the implementation testable and in scope.

5.2 Turn Enforcement via Timer or Auto-play

The use case FR-01 requires that a turn cannot stall indefinitely. The team evaluated two mechanisms for enforcing turn completion.

Decision adopted:

A Timer entity counts down from a configurable interval. If countdown reaches 0, the Controller invokes CardValidator to retrieve the lowest valid card and plays it automatically on behalf of the user, then proceeds to the BotController phase.

Alternative considered:

- Passive enforcement: no timer, the user simply plays whenever they are ready.

Rationale for rejection:

- Without a timer the game can block indefinitely, making bot turns impossible to trigger programmatically in a web environment.

Tradeoff accepted:

Auto-play may occasionally choose a card the user did not intend to play. A warning is surfaced via StatusDisplay to inform the user that an automatic play occurred.

5.3 Lead-suit Validation at the Client level

The Pitch Deck rules require that a player must follow the lead suit if they hold a card of that suit. The team chose where in the architecture to enforce this rule.

Decision adopted:

Validation is performed by a dedicated CardValidator entity before the card is moved to GameBoard. CardValidator inspects the Hand to confirm whether the player holds a card of the lead suit before accepting or rejecting the selected Card.

Alternative considered:

- Embedding validation logic directly inside the Controller.
- Defer validation to the GameBoard when the card is placed.

Rationale for rejection:

- Embedding in Controller violates the single-responsibility principle and makes the Controller harder to test and maintain as rules evolve.
- Validating at GameBoard is too late: the card has already left the Hand, requiring a rollback mechanism.

Tradeoff accepted:

A separate CardValidator concept adds a small layer of indirection; however, it makes rule changes (e.g., adding trump-suit exceptions) easier to isolate and test.

5.4 Configurable maximum score instead of fixing win condition

In the UC-4 New Game flow, the user selects a winning point threshold (50, 75, 100, or 150 points) before the game begins.

Decision adopted:

The maximum score is stored as an attribute of the Game entity (totalScore) and propagated to the GameController. The Scoreboard screen checks this value at the end of each round to determine whether the win condition has been met.

Alternative considered:

- A single hard-coded win condition (e.g., always 100 points).

Rationale for rejection:

- A fixed threshold reduces replayability and does not accommodate shorter or longer game sessions. User testing feedback indicated that session-length flexibility was a valued feature.

Tradeoff accepted:

The MaximumPoints screen adds one extra step before gameplay begins. This is mitigated by keeping the selection to four clearly labelled buttons with an immediate redirect to the Game screen on selection.

5.5 Separation of GameController (UC-4) and Controller (UC-1)

The domain models for UC-1 Card Selection and UC-4 New Game each define their own Controller concept. The team debated whether to unify these into a single top-level controller.

Decision adopted:

The two controllers are kept distinct: GameController (subsystem 2) owns game setup, score tracking, and round progression, while Controller (subsystem 1) owns per-turn card selection logic. They interact at well-defined handoff points (end of round, start of new round).

Alternative considered:

- One unified GameController responsible for both setup and in-round turn management.

Rationale for rejection:

- A single monolithic controller would carry too many responsibilities, becoming difficult to unit-test and prone to regression when either the game-setup rules or the card-play rules change independently.

Tradeoff accepted:

Two controllers introduce a coordination interface between subsystems. This is explicitly modelled via the traceability matrix (Section 2.2), ensuring the handoff contract is documented.

5.6 HTML / Browser-Based Frontend

Early in the design phase the team explored building the game as a native mobile application (React Native or a standalone desktop executable).

Decision adopted:

A web application composed of HTML screens and JavaScript, deployed and run directly in the browser. The UI described in Section 2.3 (Home, Maximum Points, Game, Scoreboard, Win/Lose screens) maps directly to HTML pages with interactive pop-ups.

Rationale:

- No installation required; the application is immediately accessible on any device with a browser.
- The team’s existing skill set centres on web technologies, reducing ramp-up time.
- HTML/CSS/JS tooling (debugging, responsive design, accessibility) is mature and well-documented.

Tradeoff accepted:

Browser-based applications cannot access certain native device features (e.g., haptic feedback, offline persistence without service workers). These features are not required by the current SRS and are

6 Pseudocode for components

Utilize the use cases to create pseudocode for components.

7 Appendices (if any)

Software component template for section 3

The template given below suggests a reasonable structure for giving a thorough description of each component described in Part 3 of the SDS. The specific information depends in part on the design approach. Your team must adapt this template to your needs and describe it in section 3.1 of your SDS.

Identification	The unique name for the component and the location of the component in the system.
Type	A module, a subprogram, a data file, a control procedure, a class, etc.
Purpose	Function and performance requirements implemented by the design component, including derived requirements. Derived requirements are not explicitly stated in the SRS, but are implied or adjunct to formally stated SDS requirements.

Function	What the component does, the transformation process, the specific inputs that are processed, the algorithms that are used, the outputs that are produced, where the data items are stored, and which data items are modified.
Subordinates	The internal structure of the component, the constituents of the component, and the functional requirements satisfied by each part.
Dependencies	How the component's function and performance relate to other components. How this component is used by other components. The other components that use this component. Interaction details such as timing, interaction conditions (such as order of execution and data sharing), and responsibility for creation, duplication, use, storage, and elimination of components.
Interfaces	Detailed descriptions of all external and internal interfaces as well as of any mechanisms for communicating through messages, parameters, or common data areas. All error messages and error codes should be identified. All screen formats, interactive messages, and other user interface components (originally defined in the SRS) should be given here.
Resources	A complete description of all resources (hardware or software) external to the component but required to carry out its functions. Some examples are CPU execution time, memory (primary, secondary, or archival), buffers, I/O channels, plotters, printers, math libraries, hardware registers, interrupt structures, and system services.
Processing	The full description of the functions presented in the Function subsection. Pseudocode can be used to document algorithms, equations, and logic.
Data	For the data internal to the component, describes the representation method, initial values, use, semantics, and format. This information will probably be recorded in the data dictionary.